

Chapter 1

Implementation of LayerIt!

In this chapter, we will explore more in depth LayerIt, the core project of this thesis. We will explore the developed architecture, and we'll be providing a broad clarification on how the system works. By the end of this chapter, the reader should better understand the value of the developed approach, as well as how it can be further developed and integrated with other systems.

1.1 Layers on Layers, within Layers

At its core, LayerIt provides a way to combine aligned visualizations of algorithmic data, audio and symbolic music representations (i.e., scores) that share the same timeline. The true value of our approach comes from the Python OO layer-based methodology that we developed in our approach. This allows not only the visualizations to be deeply customizable, but also, for implementing with other tools and workflows.

The visualizations generated by LayerIt follow a “layers on layers, within layers” philosophy: visual elements are self-contained components that can be composed into upper-level elements, which in turn can be composed into yet higher levels. ?? demonstrates this methodology.

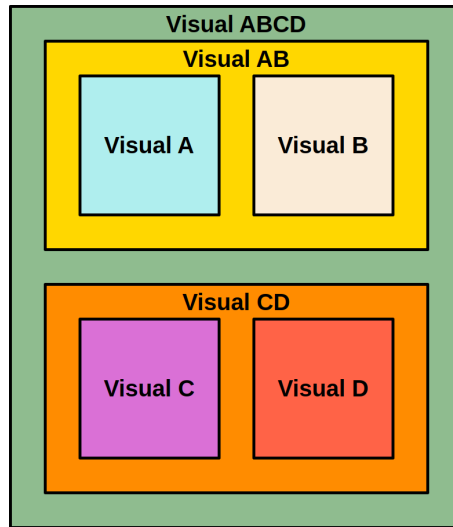


Figure 1.1: Composition of LayerIt visualization system. Each colored box represents a distinct visual element that compose higher-level visualizations.

This “layers within layers” design philosophy is applied at three different levels within the project’s codebase: the **visual level**, where visualizations are composed of aligned data, audio and musical elements; the **SVG level**, in which LayerIt organizes the XML structure of the SVG (the visual output format); to the **code level**, with it’s OO architecture of the codebase. The strength of this layer-based approach lies in the flexible foundation it provides, leaving the codebase open to future development and integration with other tools, workflows, and projects.

1.2 Generating Visuals

LayerIt was developed with the goal of generating visualizations for BA and BT tasks, therefore, the current version focuses on tools for these purposes. Hence the reason we focus so much in providing tools for audio-to-score and data alignment tools, since we believe that this is a core gap for annotation and evaluation tasks for BT and BA tasks and subsequently for MIR as a whole.

LayerIt provides tools for visualizing BT/BA relevant data, therefore in it’s current state the codebase relies on the following files to generate complete visuals:

File	Description
.wav	File with the audio recording of the performance
.mei	XML Score of the musical piece
.maps.json	Contains data that links onsets with score elements of the .mei
.svg	The score in visual format generated by ScoreWarp (requires .mei and .maps <i>a priori</i>)
.npz	File with algorithmic BT data

1.2.1 Getting the Warped Score

The warped score can be generated using the [ScoreWarp online demo](#). In it, one needs to upload the score in .mei format and the MAPS file in order to download the score in svg format.

The .mei file that contains the score in xml format. The easiest way to get any .mei score is to get the score in .musicxml format and then convert it to .mei, this can be done in multiple ways we will describe two: The first being through the [verovio online editor](#). The second, by using musescore's software that allows to then convert to .mei. In either way, it is important to get the original score in .musicxml format to be able to then convert it to .mei using the tools we described.

1.2.2 Getting the MAPS file

The MAPS file, describes the onsets of the performance and links them to each corresponding score elements of the .mei score through their @xml.id . Getting the maps file might be a bit more cumbersome, but we can describe two different ways of obtaining it:

The first is the one described in [goeblLetsScoreWarpAgain2025a](#), using the MIDI-to-MIDI matching algorithm by [nakamuraPERFORMANCEERRORDETECTION](#). However, this method requires , along side with the .mei score, the performance in midi format. Using examples from the [asap dataset](#) [peterAutomaticNoteLevelScoretoPerformance2023](#) will facilitate this process, but if a midi performance is not possible to obtain then the maps file needs to be generated independently.

The MAPS file is at it's core a .json file, and for the purposes of the current LayerIt version, it needs to provide the score's and performance onset data in the following manner:

```
{
  "obs_mean_onset": 0.8533333333,
  "xml_id": ["xliwleq1"],
  "obs_num": 1
}
```

Where:

- “*obs_mean_onset*” is the time in seconds of a given onset.
- “*xml_id*” is the @xml.id of the corresponding onset note in the .mei score. In the case that the onset has more than one note, like a chord hit, this attribute can have more than one @xml.id.
- “*obs_num*” acts as a simple counter for each detected onset.

LayerIt relies on MAPS in two complementary ways: directly for onset visualization, and indirectly through ScoreWarp, which requires MAPS to generate the warped score aligned with the timeline SVG element which constitutes the alignment reference for all overlaid data.

ScoreWarp performs best when MAPS contains dense onset annotations across the full performance. However, it can still align scores when annotations are sparse via interpolation between available onsets and their @xml.id anchor points. In practice, this allows different precision levels (i.e., downbeats, phrase boundaries, or measure-level anchors), trading annotation effort for alignment precision. Even minimal anchors (first and last onset) may be usable for short performances or segments, but intermediate alignment becomes less reliable due to interpolation error.

1.2.3 Layer Alignment Method

As of now, LayerIt provides tools for generating visualizations with BT and BA tasks in mind, however, this is not an exclusive application of the codebase. Layers (be it data, audio or scores) are simply either SVG or PNG elements that span the same or a proportional width to a given timeline, therefore, users can upload any SVG or PNG plots, given that they follow the same timeline length.

1.3 Layer Creation Logic

1.4 Conclusion

One of the main arguments of this thesis is that LayerIt also provides a framework that can support modern MIR visualizations, while offering an architecture designed for future integration with other MIR applications.

References

- [1] Brian McFee et al. *Librosa/Librosa: 0.11.0*. Version 0.11.0. Zenodo, Mar. 11, 2025. DOI: [10.5281/ZENODO.15006942](https://doi.org/10.5281/ZENODO.15006942). URL: <https://zenodo.org/doi/10.5281/zenodo.15006942> (visited on 06/22/2026).
- [2] meluron. *Modusa*. Version 6.3.7. Feb. 2026. URL: <https://github.com/meluron/modusa>.
- [3] Silvan David Peter et al. “Automatic Note-Level Score-to-Performance Alignments in the ASAP Dataset”. In: *Transactions of the International Society for Music Information Retrieval* 6.1 (June 26, 2023), pp. 27–42. ISSN: 2514-3298. DOI: [10.5334/tismir.149](https://doi.org/10.5334/tismir.149). URL: <http://transactions.ismir.net/articles/10.5334/tismir.149/> (visited on 06/07/2026).
- [4] The Matplotlib Development Team. *Matplotlib: Visualization with Python*. Version v3.10.9. Zenodo, Apr. 24, 2026. DOI: [10.5281/ZENODO.19716234](https://doi.org/10.5281/ZENODO.19716234). URL: <https://zenodo.org/doi/10.5281/zenodo.19716234> (visited on 06/22/2026).
- [5] K. Yamamoto. “Human-in-the-Loop Adaptation for Interactive Musical Beat Tracking”. In: *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. 2021.